FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Christian Dave K. Salvador
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	Oct 05, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

SOURCE CODE:

```
facts = {
    "temperature": 32,
    "is_raining": True,
    "humidity": 80,
    "has_class": True,
    "is_tired": True
}
```

```
print("========================================")
print("KNOWLEDGE REPRESENTATION")
print("========================================")
print("Knowledge Base:")
for key, value in facts.items():
    print(f"- {key}: {value}")

# PART 2 - RULE-BASED REASONING (RBR)

# Rule 1
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the air conditioner."

# Rule 2
def rule_raining(facts):
    if facts["is_raining"]:
        return "Bring an umbrella."

# Rule 3
def rule_tired(facts):
    if facts["is_tired"]:
        return "Take a short rest."

# Rule 4
def rule_class(facts):
    if facts["has_class"]:
        return "Prepare for class."

# Inference Engine
def inference_engine(facts):
    rules = [
        rule_hot,
        rule_raining,
        rule_tired,
        rule_class
    ]

    actions = []

    for rule in rules:
        result = rule(facts)

        if result:
```

```
            actions.append(result)

    return actions

# Display RBR Output
print("\n========================================")
print("RULE-BASED REASONING")
print("========================================")

actions = inference_engine(facts)

print("Rule-Based Reasoning Actions:")
for action in actions:
    print("- " + action)

# PART 3 - CASE-BASED REASONING (CBR)

# Case Base
# Contains previous student problems and their solutions.

case_base = [
    {
        "problem": {
            "temperature": 32,
            "is_raining": True,
            "is_tired": True
        },
           "solution": "Bring an umbrella, turn on the air
conditioner, and take a short rest."
    },
    {
        "problem": {
            "temperature": 25,
            "is_raining": True,
            "is_tired": False
        },
          "solution": "Bring an umbrella and wear comfortable
clothes."
    },
    {
        "problem": {
            "temperature": 35,
            "is_raining": False,
            "is_tired": True
        },
```

```
          "solution": "Turn on the air conditioner and take a
short rest."
    }
]

# New Problem
new_problem = {
    "temperature": 33,
    "is_raining": True,
    "is_tired": True
}

# Task 3B - Retrieve Step
# Calculates similarity based on matching features.

def calculate_similarity(problem1, problem2):
    matches = 0

    for key in problem1:
        if key in problem2 and problem1[key] == problem2[key]:
            matches += 1

    return matches

def retrieve_case(new_problem, case_base):
    best_case = None
    best_score = -1

    for case in case_base:
        score = calculate_similarity(
            new_problem,
            case["problem"]
        )

        if score > best_score:
            best_score = score
            best_case = case

    return best_case, best_score

# Task 3C - Reuse Step
# Reuses the solution from the most similar case.

best_case, similarity = retrieve_case(new_problem, case_base)
```

```
reused_solution = best_case["solution"]

# Task 3D - Revise Step
# In this example, the retrieved solution is accepted
# without changes.

revised_solution = reused_solution

# Task 3E - Retain Step
# Adds the new problem and solution to the case base.

new_case = {
    "problem": new_problem,
    "solution": revised_solution
}

case_base.append(new_case)

# Display CBR Output
print("\n========================================")
print("CASE-BASED REASONING")
print("========================================")

print("New Problem:")
for key, value in new_problem.items():
    print(f"- {key}: {value}")

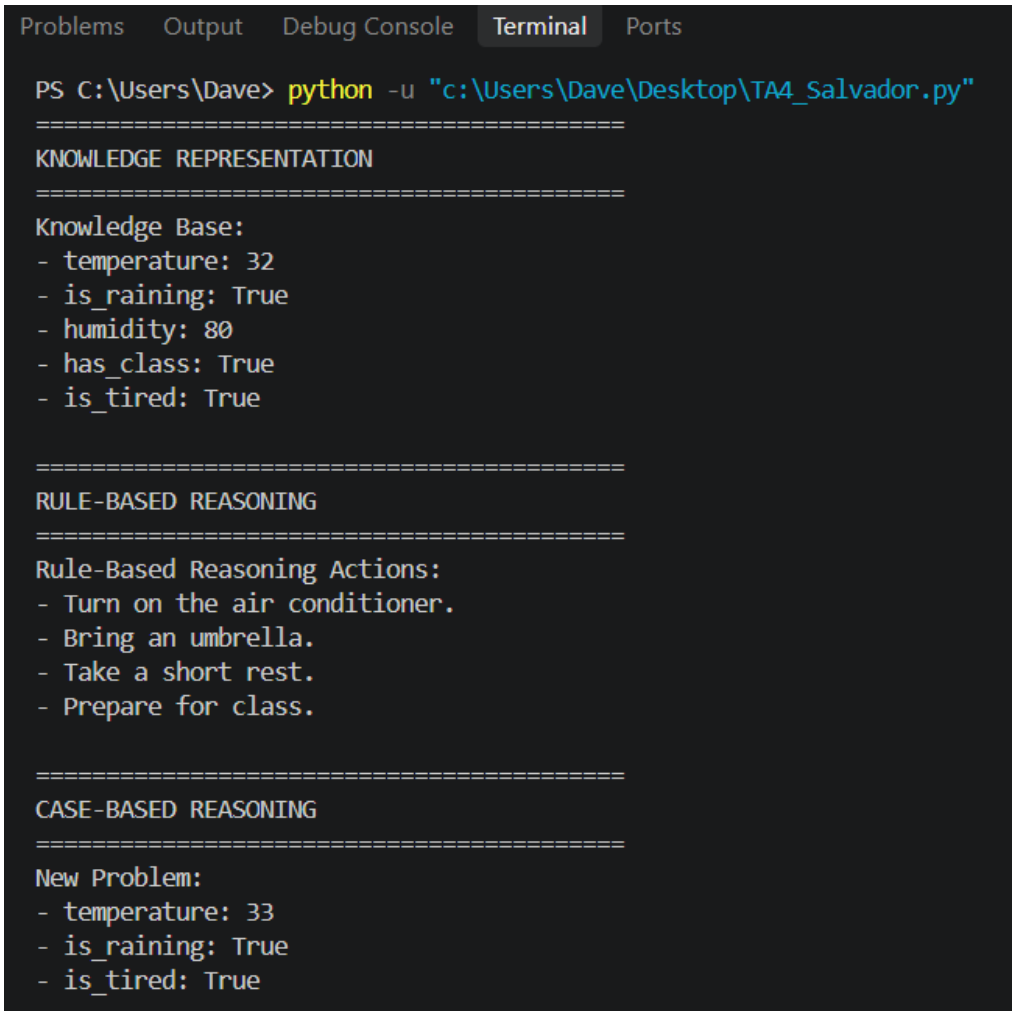
print("\nMost Similar Past Case:")
for key, value in best_case["problem"].items():
    print(f"- {key}: {value}")

print(f"\nSimilarity Score: {similarity}")
print(f"Reused Solution: {reused_solution}")
print(f"Revised Solution: {revised_solution}")

print("\nRetain Step:")
print("The new case has been added to the case base.")

print("\nUpdated Number of Cases:", len(case_base))
```

OUTPUT:



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Dave> python -u "c:\Users\Dave\Desktop\TA4_Salvador.py"
========================================
KNOWLEDGE REPRESENTATION
========================================
Knowledge Base:
- temperature: 32
- is_raining: True
- humidity: 80
- has_class: True
- is_tired: True

========================================
RULE-BASED REASONING
========================================
Rule-Based Reasoning Actions:
- Turn on the air conditioner.
- Bring an umbrella.
- Take a short rest.
- Prepare for class.

========================================
CASE-BASED REASONING
========================================
New Problem:
- temperature: 33
- is_raining: True
- is_tired: True
```

```
Most Similar Past Case:
- temperature: 32
- is_raining: True
- is_tired: True

Similarity Score: 2
Reused Solution: Bring an umbrella, turn on the air conditioner, and take a short rest.
Revised Solution: Bring an umbrella, turn on the air conditioner, and take a short rest.

Retain Step:
The new case has been added to the case base.

Updated Number of Cases: 4
PS C:\Users\Dave>
```

Short Description of the Knowledge Base and Use Case

- The knowledge base contains different facts about a student's situation, such as the temperature, whether it is raining, humidity, if the student has class, and if the student is tired. These facts are used by the system to decide what action or solution should be given. For the use case, I made the system help a student decide what to do based on their current situation. For example, if the temperature is high, the system suggests turning on the air conditioner, and if it is raining, it suggests bringing an umbrella. The CBR part also uses previous similar situations to give a possible solution to a new problem.

Comparative Reflection

- **For me, RBR was easier to implement because the rules were simple and easy to understand. I only needed to use IF-THEN conditions and check if the facts matched the rules. CBR was a little more difficult because I had to create previous cases and compare them with the new problem before getting a solution.**

  I think CBR is more flexible for unstructured problems because it can use previous experiences that are like the current problem. RBR is better when the conditions and rules are already clear and fixed. For continuously changing data, I think CBR is more useful because new cases can be added to the case base and used in the future. In real life, RBR can be used for troubleshooting systems, medical decision support, and customer support, while CBR can be used for recommendation systems, technical support, and systems that solve problems based on previous experiences.